

Liste doublement chaînée

création de la liste
ajout en tête
ajout à la fin
suppression premier élément
suppression dernier élément
recherche d'un élément

1. Structure de la liste doublement chaînée

```
#include <stdio.h>
#include <stdlib.h>

typedef struct element {
    int val;
    struct element *suivant;
    struct element *precedent;
} element;

typedef element* Liste;
```

2. Création de la liste

```
Liste creation(Liste T) {
    int n, i;
    Liste N;

    printf("Donner le nombre d'elements : ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        N = malloc(sizeof(element));

        printf("Saisir une valeur : ");
        scanf("%d", &N->val);

        N->precedent = NULL;
        N->suivant = T;

        if(T != NULL)
            T->precedent = N;

        T = N;
    }
}
```

```
    return T;
}
```

3. Affichage de la liste

```
void Affichage(Liste T)
{
    Liste P;

    P = T;

    while(P != NULL)
    {
        printf("%d ", P->val);

        P = P->suivant;
    }

    printf("\n");
}
```

4. Ajout en tête

```
void AjoutTete(Liste *T)
{
    Liste N;

    N = malloc(sizeof(element));

    printf("Entrer une valeur : ");
    scanf("%d", &N->val);

    N->precedent = NULL;

    N->suivant = *T;

    if(*T != NULL)
        (*T)->precedent = N;

    *T = N;
}
```

5. Ajout à la fin

```
void AjoutFin(Liste *T)
{
    Liste N;
    Liste P;

    N = malloc(sizeof(element));
```

```

printf("Entrer une valeur : ");
scanf("%d",&N->val);

N->suivant = NULL;

if(*T == NULL)
{
    N->precedent = NULL;
    *T = N;
}

else
{
    P = *T;

    while(P->suivant != NULL)
    {
        P = P->suivant;
    }

    P->suivant = N;

    N->precedent = P;
}
}

```

6. Suppression du premier élément

```

void SuppPremier(Liste *T)
{
    Liste P;

    if(*T == NULL)
    {
        printf("Liste vide\n");
    }

    else
    {
        P = *T;

        *T = (*T)->suivant;

        if(*T != NULL)
        {
            (*T)->precedent = NULL;
        }
    }
}

```

```

        free(P);
    }
}

```

7. Suppression du dernier élément

```

void SuppDernier(Liste *T)
{
    Liste P;

    if(*T == NULL)
    {
        printf("Liste vide\n");
    }

    else
    {
        P = *T;

        while(P->suivant != NULL)
        {
            P = P->suivant;
        }

        if(P->precedent != NULL)
        {
            P->precedent->suivant = NULL;
        }

        else
        {
            *T = NULL;
        }

        free(P);
    }
}

```

8. Recherche d'un élément

```

void Recherche(Liste T)
{
    int x;
    Liste P;

    printf("Entrer la valeur a chercher : ");
    scanf("%d",&x);
}

```

```
P = T;

while(P != NULL && P->val != x) // parcourir la liste
{
    P = P->suivant;
}

if(P == NULL)
    printf("Element non trouve\n");

else
    printf("Element trouve\n");
}
```